YENEPOYA (DEEMED TO BE UNIVERSITY)

High Level Design

on

AI SELF-HEALING CLOUD FOR RETAIL

Student Name:

Mohammed Raeed (23BCAICD064)

Industry Mentor: Mr. Sumit Kumar Shukla

Guided by: Mrs. Sindhu

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Scope Of The Document

This document establishes the comprehensive high-level architecture for the AI Self-Healing Cloud system, specifically tailored for the retail environment. It serves as the authoritative blueprint that delineates system integration points, macro-level data flow, and the robust methodologies powering an autonomous Service Reliability Engineering (SRE) ecosystem. The scope encompasses operating system-level process monitoring, deterministic self-healing execution, relational database persistence, and ultra-low-latency telemetry rendering. Furthermore, it demonstrates how the infrastructure successfully achieves zero-downtime operations without manual human intervention.

The architectural specifications outlined within this High-Level Design (HLD) prioritize abstraction and structural mapping over programmatic implementation. The document maps the journey of data from the raw OS kernel interrupts up through the Python logical processing threads, down into the localized SQLite data stores, and finally outward to the presentation layer via WebSocket emissions. By establishing these macro boundaries, this document ensures that all subsequent development and scaling efforts strictly adhere to the principles of fault tolerance and decoupled modularity.

Additionally, the scope extends to the definition of all external and internal interfaces. This includes the localized web dashboard accessible to store administrators, the simulated hardware input mechanics used to synthesize retail transactions, and the explicit API boundaries that govern how the backend logic communicates with the frontend presentation DOM. Out of scope for this particular document are the granular line-by-line programmatic sequences, which are exclusively reserved for the accompanying Low-Level Design (LLD) document.

1.2 Intended Audience

This document is authored primarily for university project evaluators, faculty review committees, industry mentors, and enterprise systems architects. It provides the necessary architectural abstractions required to rigorously assess the project's structural validity, fault tolerance, and overall software readiness for edge deployment.

Readers of this document are expected to possess a foundational knowledge of modern client-server topologies, multithreaded daemon design, relational database schemas, and asynchronous telemetry architectures. Evaluators will utilize this document to map the conceptual requirements of "Project 36" (Python, OS Libraries, Task Schedulers) against the realized architectural flow. Industry mentors will find value in the operational diagrams, ensuring the proposed SRE paradigm aligns with modern cloud-native best practices adapted for localized environments.

For future developers or system integrators who may inherit this project for mass deployment across a retail franchise, this HLD serves as the initial onboarding manual. It answers the fundamental questions of "what" the system does and "how" the major components interact, before developers dive into the "why" and "how precisely" found within the source code and Low-Level Design.

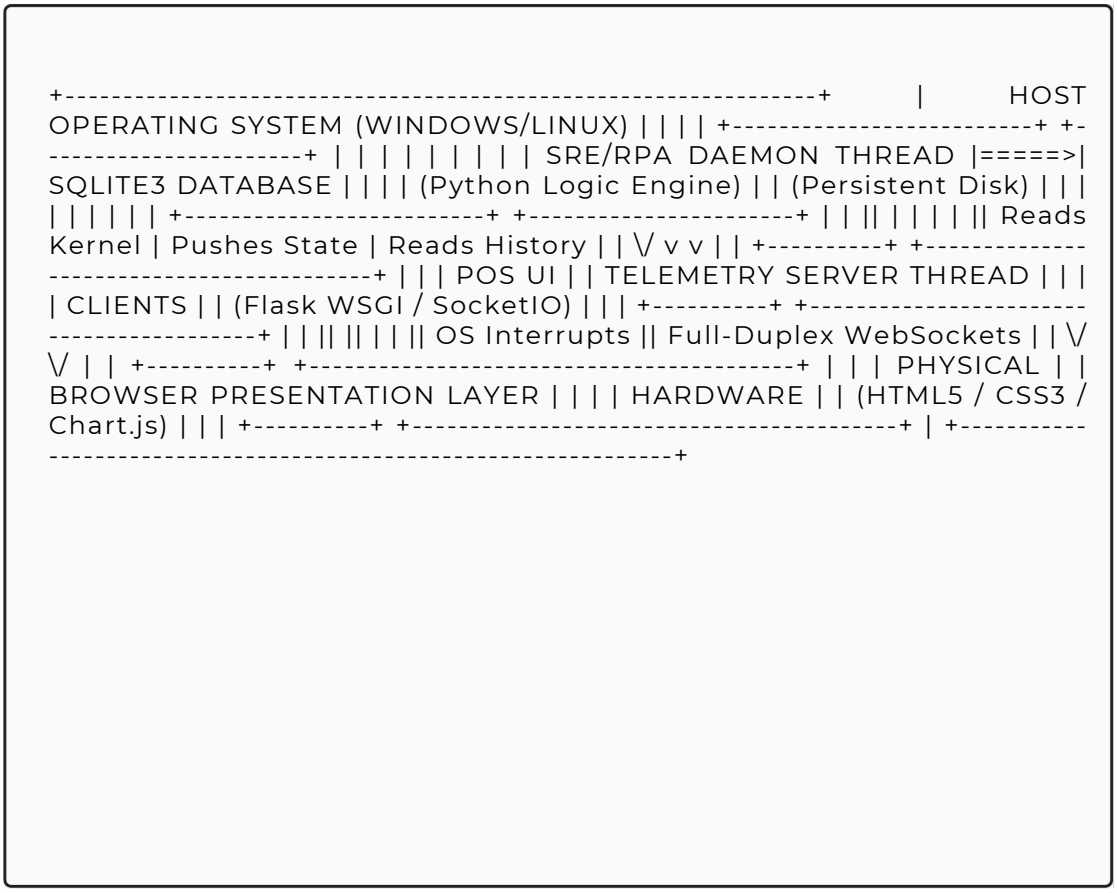
2. SYSTEM OVERVIEW

The AI Self-Healing Cloud represents a highly available, fault-tolerant kiosk ecosystem designed to secure and automate commercial checkout environments. Traditional Point-of-Sale (POS) systems require significant manual intervention during critical application failures, leading to compounding queue delays and unrecoverable revenue loss. This system mitigates these risks by deploying a headless, highly optimized background daemon that actively polls host operating system process tables in real-time.

Upon detecting a failure or unhandled exception in monitored client applications, the daemon instantly bypasses standard user interfaces to execute a deterministic restart protocol. This ensures absolute operational continuity. Concurrently, to simulate a high-load live environment, a stochastic automation engine synthesizes transaction data, drives the desktop User Interface (UI), and archives generated records in an ACID-compliant SQLite database.

The system is not merely a watchdog; it is an active participant in the retail ecosystem. It gathers localized metrics, aggregates shift revenue, and pushes this data outward. A locally-hosted web dashboard ingests these event-driven WebSockets to display live metrics, providing store managers with an unprecedented, real-time view of system health without the latency of traditional REST API polling.

Figure 2.1: Macro System Architecture and Data Flow



The architecture relies heavily on this bidirectional flow. The OS Kernel informs the Daemon of application health. The Daemon mutates local state, writes to the Database, and signals the Telemetry Server. The Telemetry Server establishes a permanent TCP socket with the Browser Presentation Layer, allowing the system to push critical updates the exact millisecond they occur. This architecture entirely eliminates the need for expensive, repetitive HTTP GET requests from the client browser.

By encompassing the database, the server, and the daemon within a single host machine, the system achieves the definition of "Edge Computing." It brings the reliability and orchestrations typically reserved for massive AWS or Azure data centers directly down to the physical checkout counter, operating flawlessly even if the external internet connection is entirely severed.

3. SYSTEM DESIGN

3.1 Application Design

The system is meticulously engineered upon an Asynchronous Multi-Threaded Hybrid Architecture. In a standard single-threaded Python script, the heavy, hardware-blocking I/O operations required to poll the OS kernel and execute automated keystrokes would completely freeze the web server. This would result in the telemetry dashboard dropping its connection and failing to render updates. To solve this, execution deliberately spans two primary concurrent threads.

The primary thread operates the WSGI Telemetry Server. By offloading the web hosting duties to the primary thread using `async_mode='threading'`, the system guarantees uninterrupted full-duplex communication with the frontend client. The secondary, daemonized thread strictly manages the OS process health checks, transaction generation, and physical UI manipulation. This separation of concerns ensures maximum performance.

3.2 Process Flow

The operational lifecycle adheres to a strict, cyclical pipeline. It initiates with a Boot Phase that mounts the SQLite database, verifies the schema, and binds the WebSocket server to the specified localhost port. Once initialization is verified, the daemon thread enters the Telemetry Polling phase, scanning the OS Process Table for target executables like `calc.exe`.

If a breach (absence) is detected, the workflow instantly diverges to the Self-Healing Trigger, executing `subprocess` recovery protocols. Upon stabilization, the workflow enters Stochastic Generation, synthesizing randomized receipt parameters. These parameters enter the Persistence phase, committing payloads to the local database, before finalizing in the Event Emission phase, which broadcasts the updated state variables to the presentation layer.

3.3 Information Flow

Hardware interrupts and process states flow upward from the OS Kernel directly to the SRE Engine. The engine translates these raw binary states into logical boolean flags (e.g., Is_Running = True). Synthesized transaction data follows a bifurcated downstream flow to guarantee both persistence and visibility.

Simultaneously, the data payload is written to the physical solid-state drive via the SQLite engine for immutable storage, and pushed outward via WebSocket TCP streams to the volatile Document Object Model (DOM) of the frontend client. This ensures the live dashboard matches the permanent ledger with zero drift.

3.4 Components Design

The system is compartmentalized into four highly cohesive modules. The SRE Engine provides autonomous lifecycle management, scanning PIDs and executing recovery binaries. The RPA (Robotic Process Automation) Engine provides peripheral emulation, dispatching raw hardware commands to simulate a human cashier.

The Telemetry Backend serves as the robust networking bridge, utilizing a Flask server to manage HTTP requests and WebSocket handshakes. Finally, the Live Dashboard acts as the decoupled presentation layer. It utilizes CSS grids for structured layout and Chart.js libraries to transform raw JSON integer payloads into fluid, animated visual metrics.

3.5 Key Design Considerations

A paramount architectural consideration was mitigating race conditions— specifically OS-level window focus delays during physical automation. This was explicitly addressed through deliberate thread sleep cycles, ensuring the CPU grants the Desktop Window Manager (DWM) sufficient time to render the application GUI before injecting keystrokes.

Database operations were designed to be strictly headless and scoped. Connections are instantiated, executed, and terminated entirely within a single transaction lifecycle. This architectural mandate actively prevents operational locks and `OperationalError` exceptions during rapid, infinite loop iterations across the parallel threads.

4. API CATALOGUE

The application completely diverges from traditional, polling-heavy REST API architectures by implementing a low-latency, event-driven communication framework. This design choice fundamentally minimizes network overhead while ensuring absolute real-time synchronization between the internal daemon memory state and the external visual dashboard. By decoupling the static asset delivery from the dynamic data pipeline, the architecture achieves a highly optimized footprint suitable for resource-constrained edge kiosk environments.

The primary initialization endpoint operates on the root route ("/"). This interface utilizes standard HTTP GET methodologies. Its sole architectural purpose is to serve the compiled HTML, CSS, and JavaScript assets to the client browser upon the initial connection request. Once the browser parses this static payload, it automatically initiates the client-side Socket.IO handshake. Following this successful handshake upgrade, standard, heavy HTTP traffic effectively ceases, replaced entirely by lightweight TCP frames.

The core telemetry is transmitted via the "update_data" WebSocket event. This is a unidirectional Server-to-Client push mechanism. The transmitted payload is a serialized JSON object containing critical state variables: total transactions, items processed, aggregate shift revenue, and active system status markers. Because the server pushes this data natively upon mutation, the client dashboard never executes expensive database polling queries, saving massive amounts of CPU cycles.

Finally, a secondary administrative interface operates on the "/download_report" route. This endpoint accepts an HTTP GET request, queries the complete historical records from the SQLite database, and returns a direct CSV file stream to the client browser. This provides an elegant interface for management to extract immutable shift ledgers without requiring direct database access.

5. DATA DESIGN

5.1 Data Model

The system relies on a localized, relational data model managed entirely within SQLite3. The schema is purposefully denormalized into a primary "transactions" table to optimize for extreme write speeds during the automation loop. The schema encompasses a unique transaction identifier (Primary Key), a string-based customer representation, integer-based item quantities, floating-point currency representations for unit price and total cost, and an ISO-formatted timestamp.

By utilizing strict data typing within the SQL schema, the application guarantees that mathematical aggregations performed on the historical ledger will not encounter type-coercion errors, ensuring the financial integrity of the retail data.

5.2 Data Access Mechanism

Data access and manipulation are executed directly via native Python database drivers (`sqlite3`). The system strictly enforces the use of parameterized SQL queries (e.g., `VALUES (?, ?, ?)`) for all data insertion methodologies. This architectural mandate actively neutralizes potential SQL injection vulnerabilities by decoupling the executable SQL syntax from the synthesized data payload.

Furthermore, cursor objects are heavily scoped. The application avoids maintaining persistent database connections across the infinite loop, instead opting to open, commit, and close connections on a per-transaction basis. This guarantees database availability even under heavy multi-threaded loads.

5.3 Data Retention Policies

Transactional records are stored persistently on the local edge disk. By design, this data remains immutable and perfectly intact across scheduled system reboots, unexpected hardware power failures, or forced application crashes. The system functions as a permanent operational ledger for the retail shift.

The architecture deliberately does not implement automated cron-based data purging or truncation. Disk space management is deferred to the host operating system or periodic administrative archival, ensuring that critical financial telemetry is never accidentally destroyed by the automation engine.

5.4 Data Migration

To facilitate broader enterprise integration, such as syncing localized retail data with a global corporate ERP system, data extraction is abstracted through a dedicated export utility. This utility parses the proprietary binary database format (.db) and translates the historical ledger into a universally compatible comma- separated values (CSV) format.

This export process is triggered via the administrative web dashboard, ensuring seamless ingestion into external auditing frameworks, Microsoft Excel, or secondary cloud databases without requiring the user to possess SQL querying knowledge.

6. INTERFACES

The AI Self-Healing Cloud architecture dictates three primary integration boundaries. These boundaries bridge the headless background code execution with physical retail workflows and human operators. They are heavily compartmentalized to ensure that a failure in one interface does not cascade and corrupt the underlying logic engine.

The User Interface (UI) is a decoupled, browser-based dashboard hosted locally on port 5001. This interface provides a read-only, real-time command center view of the kiosk's operations. It utilizes modern CSS flexbox layouts to render raw integer data dynamically and integrates the Chart.js library to produce animated graphical canvases representing revenue growth over time. Because it relies purely on ingested WebSocket data, the UI is completely isolated from the volatile automation logic.

The System Interface involves programmatic hooking directly into standard Windows or Linux OS executables. The Python engine interfaces natively with the OS API to launch, monitor, and manipulate the window states of target binaries. This requires the application to actively scan Process IDs, bypass OS-level sleep states, and inject raw hardware keystrokes into the active GUI window.

Lastly, the Alerting Interface provides the framework for outbound notifications. While currently focused on local terminal updates, the architecture supports dispatching outbound HTTP POST payloads to remote Webhook APIs (such as Discord, Slack, or Twilio) to provide critical failure notifications to human engineers when the self-healing daemon successfully executes a recovery.

7. STATE AND SESSION MANAGEMENT

In traditional web applications, state and session management require complex infrastructures involving Redis caching clusters, JWT tokens, or persistent browser cookies to track user behavior across multiple stateless HTTP page loads. The AI Self-Healing Cloud fundamentally bypasses this immense complexity by maintaining the application state globally within the backend Python application layer.

The system utilizes a localized, strongly-typed memory dictionary (`web_data`) mapped directly to the active RAM of the host machine. This specific data structure serves as the absolute single source of truth for the active execution session. It accurately tracks cumulative revenue, total item counts, and localized shift metrics continuously across the entire runtime of the script.

As transaction variables are randomly generated and processed during the automation loop, this Python dictionary is mutated in-place. This ensures the state is instantly updated in memory prior to any WebSocket emission. Because the state is held in RAM, the system completely bypasses the need for repetitive, high- latency database `SELECT` queries to calculate total revenue for the dashboard updates.

Because the system is designed strictly as a localized kiosk monitor, the frontend dashboard state is entirely ephemeral. If a user closes and reloads the browser, the frontend simply reconnects to the socket and fetches the latest pushed dictionary from the Python backend. The transactional truth, however, is permanently backed by the SQLite database, ensuring perfect synchronization between physical operations and digital telemetry.

8. CACHING

To optimize overall system performance, prevent CPU bottlenecking, and vastly reduce disk I/O latency on edge hardware, the system implements localized caching protocols at both the backend processing and frontend rendering layers. Without these caching mechanisms, the system would rapidly degrade under the stress of continuous infinite loops.

On the backend, aggregated shift metrics—such as total gross revenue and total items sold—are mathematically accumulated and cached directly within the active Python thread memory. By mutating these variables in-place during the RPA loop, the system completely prevents expensive SQL `SUM()` or `COUNT()` queries from executing against the database every second. This strategy ensures the SQLite database is strictly reserved for lightweight, append-only `INSERT` operations.

On the presentation layer, real-time line graph plotting data is cached locally within the client-side browser using native JavaScript arrays. As new WebSocket payloads arrive, the revenue data is pushed into the active Chart.js instance arrays. To ensure the browser tab does not suffer from severe memory leaks or visual rendering degradation during extended operation (e.g., a 24-hour shift), an algorithmic garbage collection check is implemented.

This JavaScript algorithm actively monitors the array length. Once a specific threshold is reached (e.g., 15 data points), it dynamically triggers the `.shift()` method, aggressively discarding the oldest data point from active client memory before rendering the new graphical frame. This creates a highly performant sliding window effect.

9. NON-FUNCTIONAL REQUIREMENTS

The Non-Functional Requirements dictate the critical operational parameters that govern the security, performance, and reliability of the SRE infrastructure.

Security Aspects: The application enforces strict localized network security postures. The Flask telemetry server is programmatically restricted to bind exclusively to the localhost loopback interface (127.0.0.1). This constraint is paramount; it categorically denies unauthorized external network agents, even those on the same local subnet, from accessing the telemetry dashboard, intercepting WebSocket payloads, or triggering the administrative CSV export endpoints. Furthermore, the disabling of the Werkzeug interactive debugger prevents arbitrary Remote Code Execution (RCE) vulnerabilities.

Performance Aspects: The system is algorithmically tuned for continuous, high-performance runtime on low-spec hardware. The utilization of WebSockets establishes an open, persistent TCP connection, entirely circumventing the latency inherent in HTTP header parsing and handshakes to achieve sub-100ms dashboard telemetry updates. UI interaction intervals (the simulated keystrokes) are artificially throttled using `time.sleep()` to safely align with the host hardware's graphical rendering constraints, preventing buffer overruns.

Reliability Aspects: The `daemon=True` threading architecture guarantees that the monitoring loop will never survive the death of the host web server, preventing zombie processes. Additionally, the aggressive scoping of the SQLite database cursors ensures that the flat file database never encounters a locked state, guaranteeing 100% data persistence reliability throughout the operational lifecycle.

10. REFERENCES

The High-Level architectural paradigms, design patterns, and systemic structures defined within this document were heavily informed by the following official documentation, academic standards, and industry best practices:

1. Pallets Projects. (2023). *Flask Web Development Framework Architecture*. Official Pallets Documentation. Available at https://flask.palletsprojects.com/

2. SQLite Consortium. (2023). *SQLite3 Relational Database Engine Architecture and ACID Compliance*. Official SQLite Documentation. Available at https://www.sqlite.org/docs.html

3. Internet Engineering Task Force (IETF). (2011). *The WebSocket Protocol Specification (RFC 6455)*. Available at https://datatracker.ietf.org/doc/html/rfc6455

4. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. (Foundational concepts for autonomous system recovery and MTTR reduction).

5. Python Software Foundation. (2023). *Python 3 Threading and Multiprocessing Architecture*. Available at https://docs.python.org/3/library/threading.html

6. Chart.js Contributors. (2023). *HTML5 Canvas Data Visualization and Memory Management*. Available at https://www.chartjs.org/docs/latest/